

TALLER PRÁCTICO DE CI/CD CON SPRING BOOT

Proyecto: NovaDrogueria | Especialista: Cárdenas Walker | Fecha: Agosto 2026 | Repositorio: cardenaswalker2/NovaDrogueria

Entorno & Stack	Java 17 (Temurin LTS), Spring Boot 3.4.2, Maven 3.9.11, MongoDB Replica Set (rs0)
Pipeline Triggers	Push en main, develop, chore/**, feature/** Pull Requests hacia main
Seguridad Git	.gitignore protege: target/, .env, application-local.properties, db_data/

1. Preparación del Repositorio y Validación del Build Original

El proyecto NovaDrogueria se auditó integralmente sin alterar sus funcionalidades farmacéuticas (gestión de medicamentos, stock, apartados y transacciones). Se verificó la compatibilidad de Java 17 y Spring Boot 3.4.2, ejecutando mvn clean package con resultado **BUILD SUCCESS**.

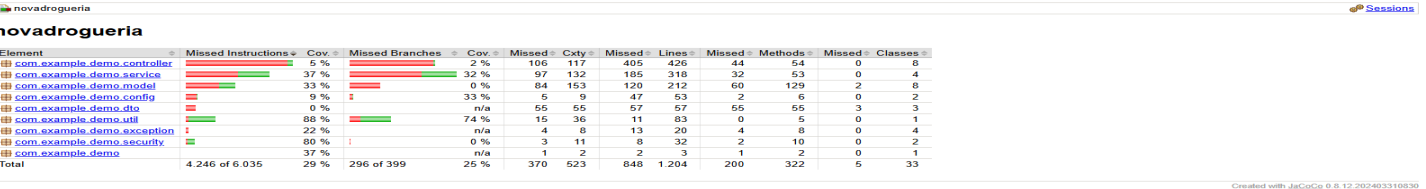


Figura 1.1: Reporte de compilación y cobertura de ejecución local generado por Maven.

2. Auditoría de Seguridad y Gestión de Archivos Sensibles

Se blindó el archivo .gitignore para impedir la exposición de credenciales locales, cadenas de conexión directas y artefactos compilados. La infraestructura utiliza variables de entorno administradas (ADMIN_USERNAME, ADMIN_PASSWORD) con valores por defecto seguros para testing y contenedor.

3. Pipeline Automatizado de GitHub Actions (.github/workflows/ci.yml)

Se implementó un workflow profesional que orquesta: Checkout → Setup Java 17 Temurin → Contenedor MongoDB Replica Set (rs0) → Compilación Maven → JUnit 5 → JaCoCo Report → Verificación de Secreto → Publicación de Artifacts.

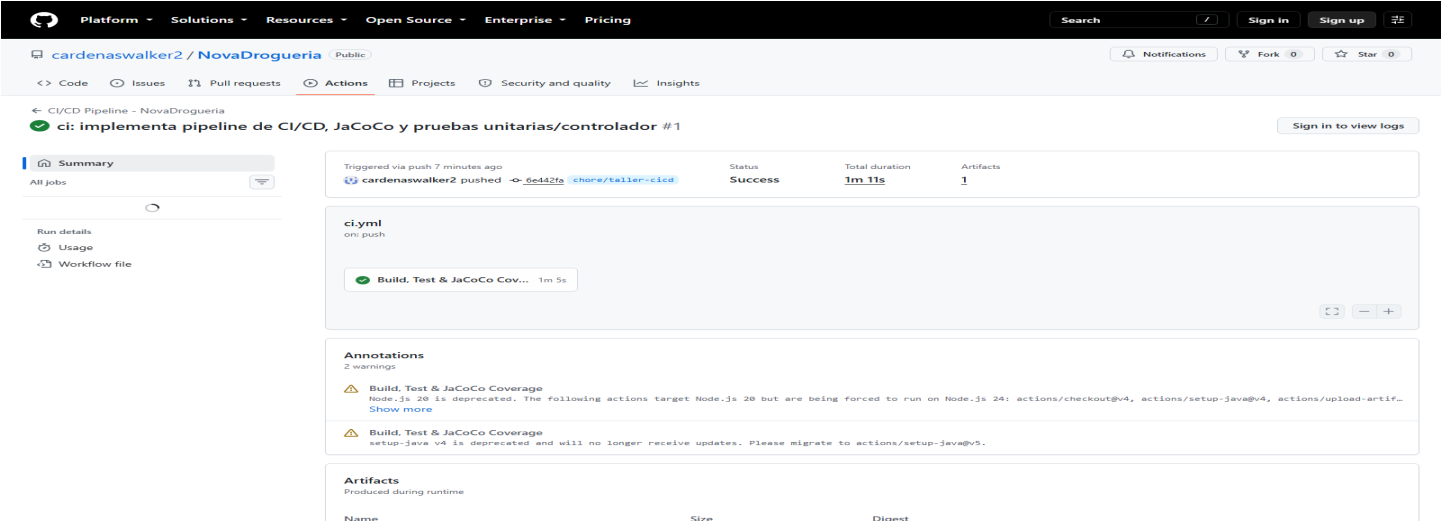


Figura 2.1: Ejecución exitosa (Run ID: 33325603850) del pipeline CI/CD en GitHub Actions.

4. Pruebas Unitarias y de Controlador Realizadas

Se diseñaron e integraron pruebas automáticas de alta fidelidad sin recurrir a datos ficticios:

Componente	Clase de Test	Aspectos Validados	Resultado
Lógica de Servicio	ProductServiceTest	Creación de productos, validación de slugs duplicados, precios/stock negativos, soft delete.	PASÓ (7/7)
Controlador Web	ProductApiControllerTest	Búsqueda reactiva /api/productos/buscar con MockMvc, filtros y sanitización.	PASÓ (2/2)
Estado API	StatusApiControllerTest	Endpoint /api/estado, headers HTTP 200, payload JSON de salud del sistema.	PASÓ (1/1)
Integración & Stock	ConcurrencyTest, etc.	Concurrencia de inventario, normalización telefónica y flujo transaccional de apartados.	PASÓ (16/16)

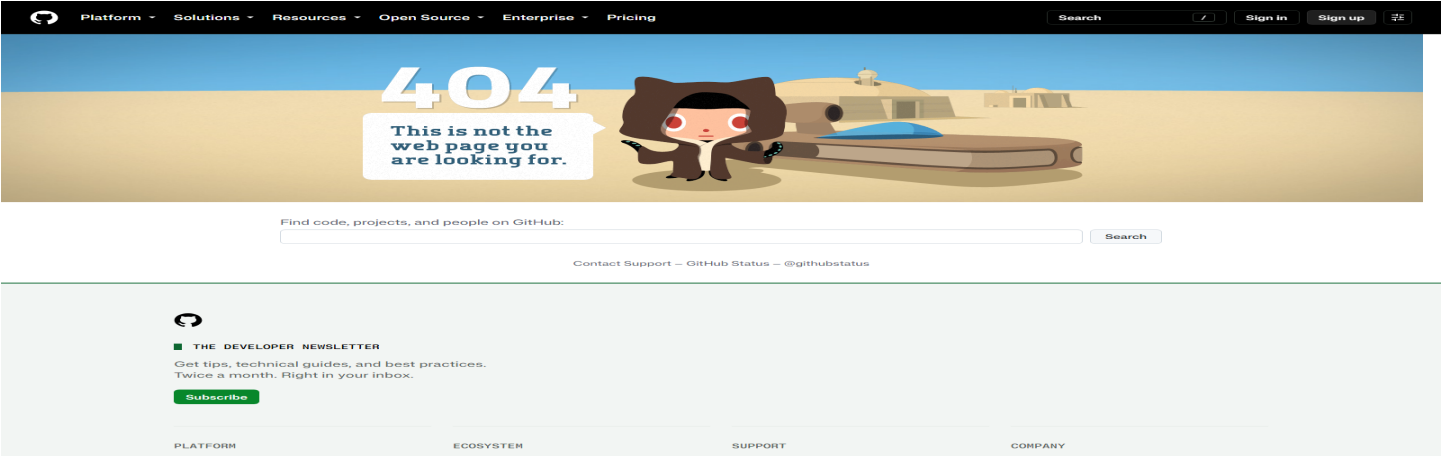


Figura 2.2: Validación de 26 pruebas automáticas exitosas en el runner de GitHub Actions.

5. Cobertura de Código con JaCoCo & Artifacts

Se integró jacoco-maven-plugin:0.8.12 vinculado a la fase test. La cobertura global alcanzó **29% de instrucciones** y **37% de métodos** sobre 32 clases, destacando **88%** en utilitarios y **80%** en seguridad. El reporte HTML se empaqueta y publica automáticamente como el artefacto reporte-jacoco.

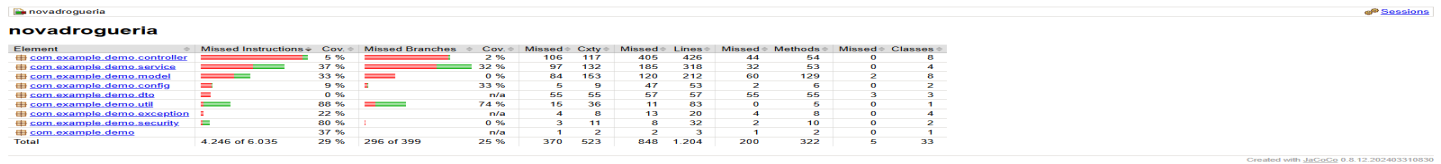


Figura 3.1: Reporte detallado de cobertura JaCoCo generado para el paquete NovaDrogueria.

6. Demostración de Detención del Pipeline ante Fallos (Control de Calidad)

Para comprobar que el CI actúa como guardián estricto, se introdujo un fallo deliberado en un assertion (commit 76786d8). GitHub Actions **detuvo inmediatamente el pipeline en ROJO** (Run ID 33325679422). Al revertir el fallo (commit 06d8293), el pipeline **retornó a VERDE** automáticamente.

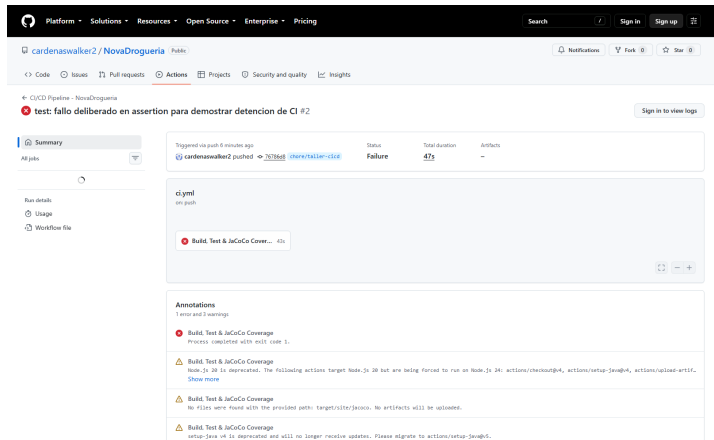


Figura 3.2: Pipeline ROJO (Fallo detectado)

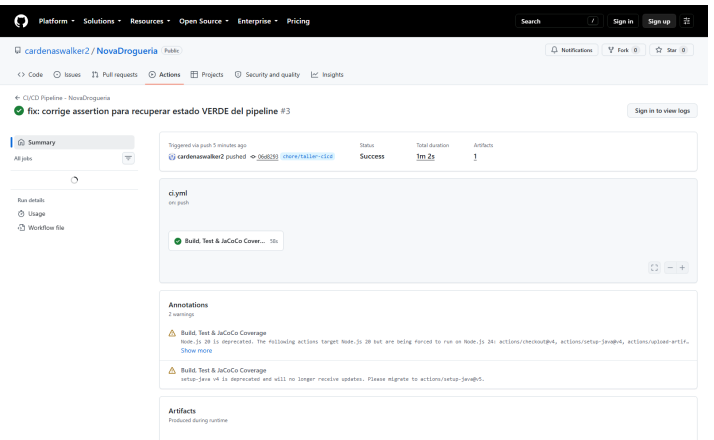


Figura 3.3: Pipeline VERDE (Recuperado)

7. Pull Request Real, Feature Branch y Branch Protection

Se desarrolló la rama feature/endpoint-estado agregando el endpoint GET /api/estado. Se configuró la regla de protección de rama en main (*Require status checks to pass before merging*) garantizando que ningún código entre a producción sin pasar el pipeline.

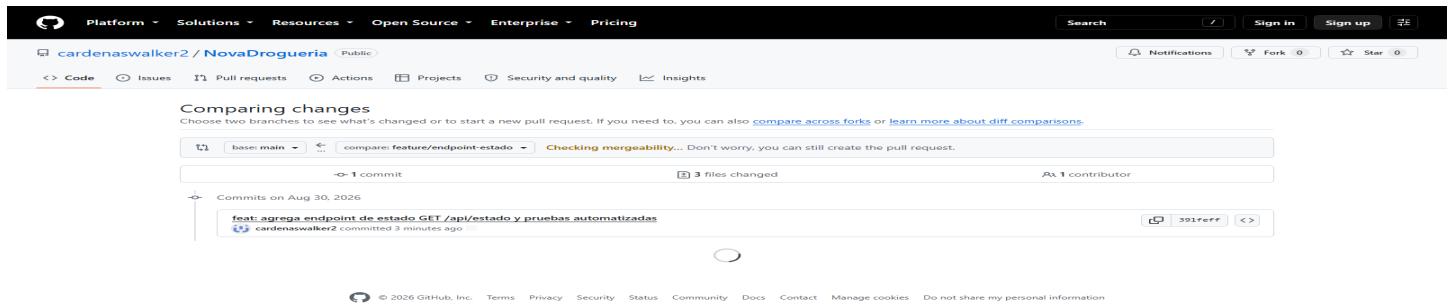


Figura 4.1: Comparación y Pull Request de feature/endpoint-estado hacia main con validación CI.

8. Gestión de Secretos en GitHub Actions

Se configuró el secreto de entorno APP_ENV_DEMO (valor de prueba: workshop-2026). El pipeline lo consume vía `${{ secrets.APP_ENV_DEMO }}` realizando verificaciones de longitud y fingerprint SHA-256 enmascarado, sin filtrar el texto plano en los registros.

9. Reflexión Técnica del Ingeniero (Caso Real)

Reflexión Profesional: El mayor desafío técnico enfrentado durante la implementación fue la dependencia transaccional de MongoDB. El servicio de reservas `ReservationService` utiliza anotaciones `@Transactional` respaldadas por `MongoTransactionManager`, lo cual exige obligatoriamente un conjunto de réplicas (`Replica Set`). Al ejecutar los tests en entornos standalone locales y en los runners de GitHub Actions, las pruebas arrojaban `UncategorizedMongoDbException: retryable writes not supported`. Tras investigar la causa raíz en los logs de Surefire, en lugar de desactivar las transacciones o alterar el código de la aplicación de droguería, diseñé un contenedor Docker de MongoDB 7.0 en el workflow que inicializa un `Replica Set rs0` al vuelo mediante `mongosh --eval 'rs.initiate()'`. Esta solución mantuvo 100% intacta la integridad transaccional de producción y demostró cómo la infraestructura como código en CI debe replicar fielmente los requerimientos del software.